

Site and row model

Legalization snaps cells to a discrete grid

The idea

- Every cell sits on exactly one row with a lower-left coordinate aligned to site pitch one
- Width tells you how many consecutive sites the rectangle covers
- The golden packing is legal

Pseudocode

- Pseudocode is the written sketch of the algorithm before you code it
- For this module the sketch is the site and row model itself
- Outputs are the rules every legal packing must obey
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- Read the sketch as a contract
- Every cell lower-left sits on a site and a row bottom
- Width tells how many consecutive sites the rectangle covers
- Our teaching chip is twelve by six with three rows

Algorithm sketch — try these

```
INPUT: chip W×H, siteW, rowH, rows Y[], widths w[c]
OUTPUT: legal coordinate rules
for each cell c:
    x multiple of siteW; y in Y[]
    occupies [x, x+w[c]) × [y, y+rowH)
GOLDEN: W=12 H=6 siteW=1 rowH=2 Y={0,2,4}
widths A–D=2 E–F=1 (total 10 ≤ 12)
```

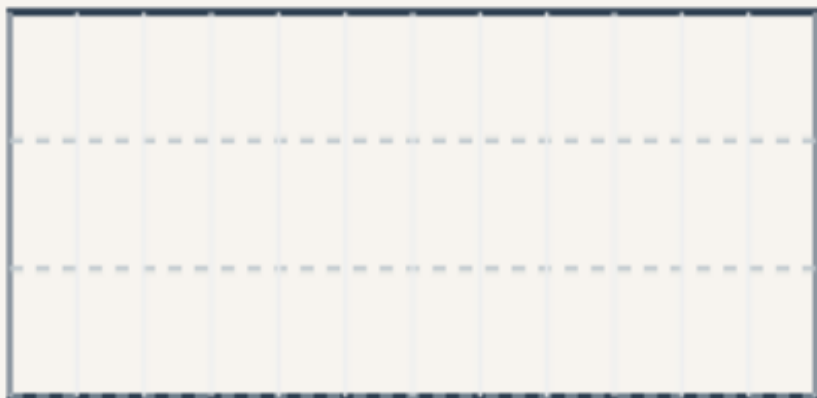
Twelve-by-six chip, three rows

SITE & ROW MODEL

Twelve-by-six chip, three rows

Step 1 / 5 · empty-chip · module01-01-site-row-model

chip 12×6 · 3 rows · site 1



Explanation

Legalization lives on a discrete canvas: chip width twelve sites, height six (three rows of height two). Row bottoms sit at y equals zero, two, and four. Every movable cell must land on a row and align to site pitch one.

- $\text{CHIP_W} = 12 \cdot \text{CHIP_H} = 6$
- Row bottoms: $y = 0, 2, 4$
- Lower-left origin for cell coordinates

```
chip: 12×6  
rows: 3  
rowH: 2
```

Twelve-by-six chip, three rows

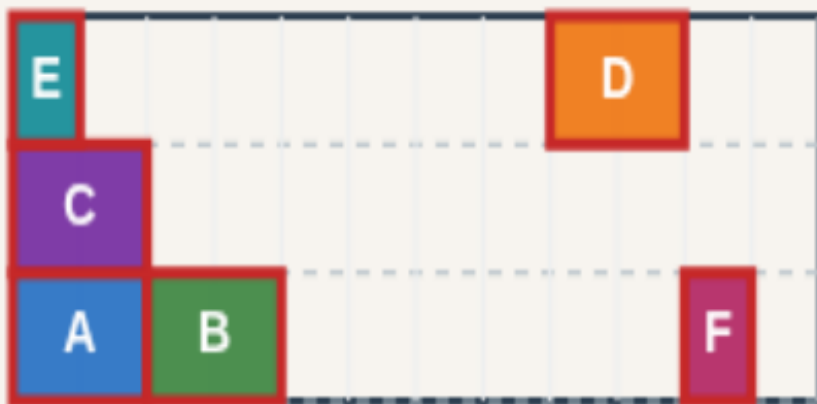
Cell widths A-D = 2, E-F = 1

SITE & ROW MODEL

Cell widths A-D = 2, E-F = 1

Step 2 / 5 · cell-widths · module01-01-site-row-model

chip 12×6 · 3 rows · site 1



Explanation

Six cells A through F occupy ten sites total—two sites each for A–D and one each for E and F. Width drives how many consecutive sites a rectangle spans when you pack a row.

- Widths are integers in site units
- Total width $10 \leq \text{chip } 12$
- Same six-cell toy across all labs

A–D width: 2
E–F width: 1
total width: 10

Cell widths A-D = 2, E-F = 1

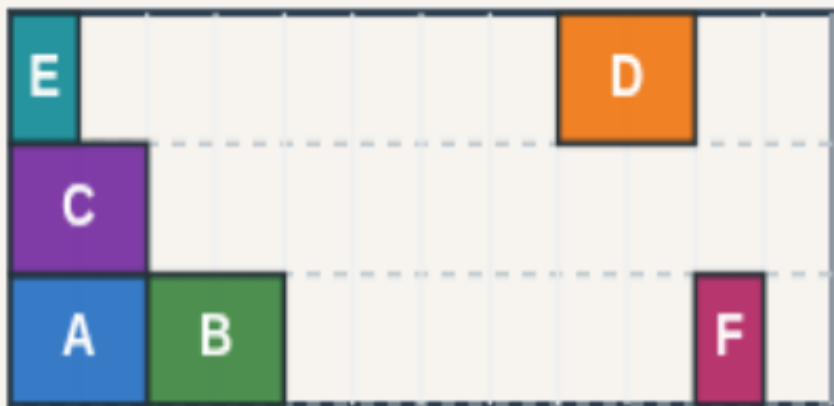
Site pitch equals one

SITE & ROW MODEL

Site pitch equals one

Step 3 / 5 · site-pitch · module01-01-site-row-model

chip 12×6 · 3 rows · site 1



Explanation

Each vertical tick is one site. Legal x coordinates are integers; a width-two cell at x equals four covers sites four and five on its row. The golden packing shows legal site alignment.

- `SITE_W = 1`
- x must be integer-aligned
- Row lines dashed at `y = 0, 2, 4`

```
site pitch: 1  
rows: 0, 2, 4
```

Site pitch equals one

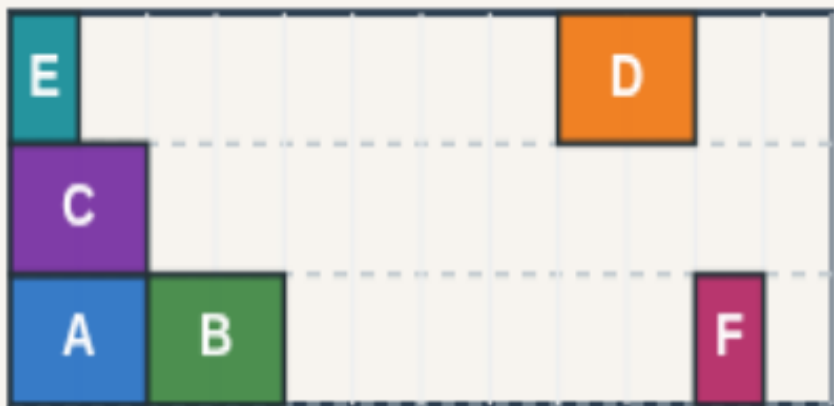
Golden legal packing

SITE & ROW MODEL

Golden legal packing

Step 4 / 5 · golden-pack · module01-01-site-row-model

chip 12×6 · 3 rows · site 1



Explanation

The reference legal layout spreads A and B on row zero, C on row one, and E plus D on row top. No overlap, every cell on-row and site-aligned—this is the teaching golden.

- Row 0: A@0, B@2, F@10
- Row 2: C@0
- Row 4: E@0, D@8

```
legal: true  
HPWL: 50  
reason: ok
```

Golden legal packing

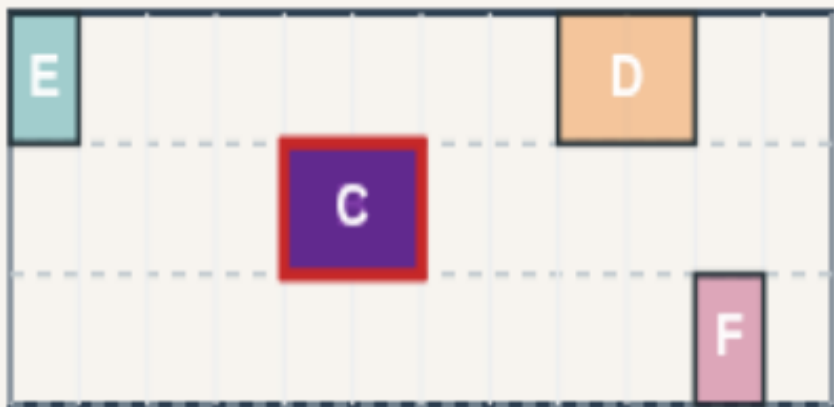
Contrast: illegal overlap stack

SITE & ROW MODEL

Contrast: illegal overlap stack

Step 5 / 5 · overlap-contrast · module01-01-site-row-model

chip 12x6 · 3 rows · site 1



Explanation

The overlap seed stacks A, B, and C at (4, 2) on the middle row—same width-two cells fighting for the same sites. Legality fails before you even discuss wirelength.

- A,B,C @ (4,2) stacked
- First failure: overlap A/B
- Legalization must repair this

legal: false
reason: overlap A/B
Next: legality metrics

Contrast: illegal overlap stack

Browser lab track

- In the browser lab track, open the **site-row-model** lab from the tools shelf
- Open the interactive lab
- Reveal golden is study-only
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse `tiny\_legal.json`, run the algorithm with deterministic coordinates
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module